MIPT & YSDA · AUTUMN 2026

Deep Generative Models

Lecture 8

Roman Isachenko

Moscow Institute of Physics and Technology
Yandex School of Data Analysis

Recap of Previous Lecture

Fix a schedule $0 < \beta_t < 1$. The noises $\boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{I})$ are independent across t and of $\mathbf{x}_0$.

$$\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_t, \quad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{I})$$

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

Exact Conditional Distribution

Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \prod_{s=1}^{t}(1-\beta_s)$. Then

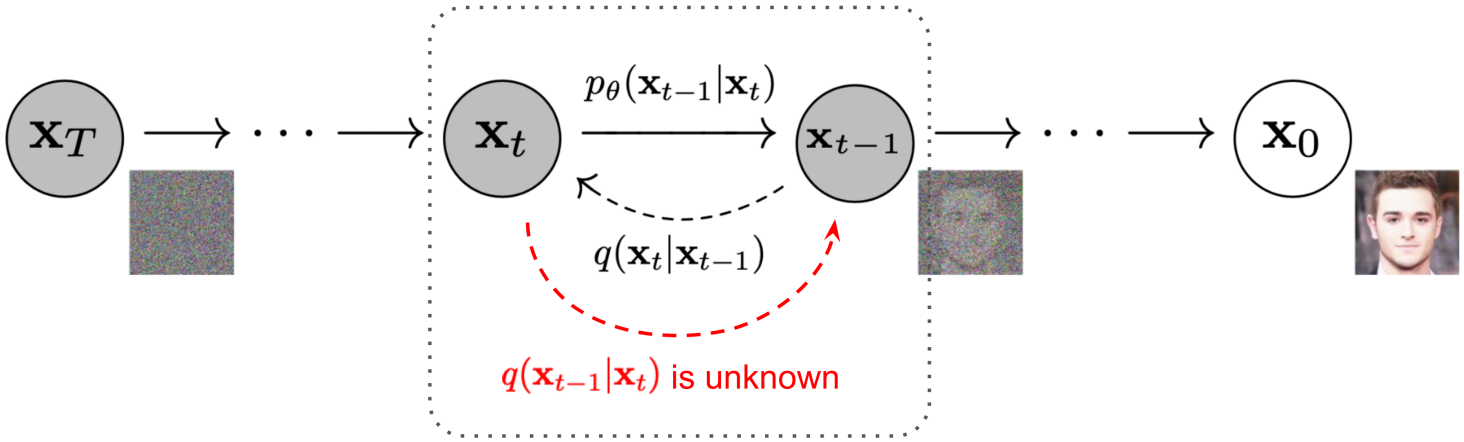
$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I})$$

Connection to NCSN: $\mathbf{x}_t = \mathbf{x}_0 + \sigma_t\boldsymbol{\epsilon}$ adds Gaussian noise; DDPM also scales the signal by $\sqrt{\bar{\alpha}_t}$.

For finite T, choose $\bar{\alpha}_T \ll 1$ so the residual signal $\sqrt{\bar{\alpha}_T}\mathbf{x}_0$ is negligible; then $q(\mathbf{x}_T) \approx \mathcal{N}(0, \mathbf{I})$.

Sohl-Dickstein J. Deep Unsupervised Learning using Nonequilibrium Thermodynamics, 2015
Chan S. Tutorial on Diffusion Models for Imaging and Vision, 2024
Song Y. et al. Generative Modeling by Estimating Gradients of the Data Distribution, 2019

Recap of Previous Lecture



Reverse Process (Ancestral Sampling)

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)}$$
$$\approx p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}\left(\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \boldsymbol{\sigma}^2_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right)$$

If β_t is sufficiently small, $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is approximately Gaussian.

Recap of Previous Lecture

Forward Process

1. $\mathbf{x}_0 \sim p_{\text{data}}(\mathbf{x}_0)$
2. $\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_t$
3. $q(\mathbf{x}_T) \approx \mathcal{N}(0, \mathbf{I})$

Reverse Process

1. $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$
2. $\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \boldsymbol{\sigma}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\boldsymbol{\epsilon}_t$
3. $\mathbf{x}_0 \sim p_{\boldsymbol{\theta}}(\mathbf{x}_0) \approx p_{\text{data}}(\mathbf{x}_0)$

SNR and Log-SNR

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

$$\text{SNR}(t) = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t}.$$

$$\lambda_t = \log \text{SNR}(t), \qquad \bar{\alpha}_t = \frac{1}{1+e^{-\lambda_t}}.$$

$\lambda_t > 0$: signal dominates; $\lambda_t = 0$: equal scales; $\lambda_t < 0$: noise dominates.

Weng L. What are Diffusion Models?, blog post, 2021
Kingma D. et al. Variational Diffusion Models, 2021

Recap of Previous Lecture

Forward process: with a suitable noise schedule, $q(\mathbf{x}_T) \approx \mathcal{N}(0, \mathbf{I})$ as the residual signal becomes negligible:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\,\mathbf{x}_{t-1}, \beta_t\,\mathbf{I});$$
$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0, (1-\bar{\alpha}_t)\,\mathbf{I}).$$

Reverse process refers to an intractable distribution that can be approximated by a normal distribution (with unknown parameters) for small β_t:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)} \approx \mathcal{N}\left(\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \boldsymbol{\sigma}^2_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right)$$

Conditioned reverse process is a normal distribution with known parameters, describing how to denoise a noisy image $\mathbf{x}_t$ when we know the clean image $\mathbf{x}_0$.

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t\,\mathbf{I})$$

Recap of Previous Lecture

- $\mathbf{z} = (\mathbf{x}_1, \ldots, \mathbf{x}_T)$ represents the latent variables.
- **Fixed encoder:** the noise schedule determines the variational posterior:

$$q(\mathbf{z}|\mathbf{x}) = q(\mathbf{x}_1, \ldots, \mathbf{x}_T|\mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1}).$$

- **One-step decoder and learned Markov-chain prior:**

$$p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z}) = p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1); \quad p_{\boldsymbol{\theta}}(\mathbf{z}) = \prod_{t=2}^{T} p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) \cdot p(\mathbf{x}_T)$$

> **The encoder has no trainable parameters; only θ is optimized.**

Das A. An Introduction to Diffusion Probabilistic Models, blog post, 2021
Ho J. Denoising Diffusion Probabilistic Models, 2020

Recap of Previous Lecture

Diffusion ELBO (Fixed Encoder)

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})}{q(\mathbf{z}|\mathbf{x})} = \mathcal{L}_{\boldsymbol{\phi},\boldsymbol{\theta}}(\mathbf{x}) \to \max_{\boldsymbol{\theta}}$$

$$\mathcal{L}_{\boldsymbol{\phi},\boldsymbol{\theta}}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) - D_{\mathrm{KL}}\big(q(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T)\big) - \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)\|p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big)}_{\mathcal{L}_t}$$

Reconstruction: the decoder recovers $\mathbf{x}_0$ from $\mathbf{x}_1$.
For a continuous Gaussian decoder with fixed covariance $\sigma_1^2\mathbf{I}$, $\sigma_1^2 > 0$:

$$-\mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) = \frac{1}{2\sigma_1^2} \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \big\|\mathbf{x}_0 - \boldsymbol{\mu}_{\boldsymbol{\theta},1}(\mathbf{x}_1)\big\|^2 + \text{const}.$$

Prior matching: constant in $\boldsymbol{\theta}$, since q and $p(\mathbf{x}_T) = \mathcal{N}(0, \mathbf{I})$ are fixed.

Intermediate denoising: match each learned reverse step to its known conditioned target, $t \geq 2$.

Das A. An Introduction to Diffusion Probabilistic Models, blog post, 2021
Ho J. Denoising Diffusion Probabilistic Models, 2020

Recap of Previous Lecture

Gaussian Reverse Steps

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\big(\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}\big), \qquad p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}\big(\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \tilde{\beta}_t \mathbf{I}\big).$$

For $t \geq 2$, fixing the model covariance to the target covariance gives

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left[\frac{1}{2\tilde{\beta}_t} \left\| \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) \right\|^2 \right].$$

Known Regression Target

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} \mathbf{x}_0,$$

$$\tilde{\beta}_t = \frac{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}.$$

Learn the reverse mean by weighted regression onto the conditioned forward-process mean.

Recap of Previous Lecture

$$-\mathcal{L}_{\boldsymbol{\phi},\boldsymbol{\theta}}(\mathbf{x}_0) = -\mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) + D_{\mathrm{KL}}\big(q(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T)\big) + \sum_{t=2}^{T} \mathcal{L}_t.$$

Training

1. Sample $\mathbf{x}_0 \sim p_{\text{data}}$, $t \sim \text{Uniform}\{2, \dots, T\}$.
2. Sample $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$; construct $\mathbf{x}_1$, $\mathbf{x}_t$.
3. Minimize $\hat{\mathcal{L}}$ by SGD: the omitted prior KL is independent of $\boldsymbol{\theta}$.

$$\mathbf{x}_s = \sqrt{\bar{\alpha}_s}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_s}\boldsymbol{\epsilon}, \quad s \in \{1, t\}.$$

$$\hat{\mathcal{L}} = -\log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) + \frac{T-1}{2\tilde{\beta}_t}\left\|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right\|^2.$$

$T-1$: uniform timestep average $\to$ sum.

Sampling

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
2. Denoise $\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sqrt{\tilde{\beta}_t}\boldsymbol{\epsilon}_t$.
3. Return the mean $\hat{\mathbf{x}}_0 = \boldsymbol{\mu}_{\boldsymbol{\theta},1}(\mathbf{x}_1)$.

For $t = T, \dots, 2$, sample the **reverse kernel**:

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}\big(\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t), \tilde{\beta}_t\mathbf{I}\big).$$

Use fresh $\boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{I})$ at each step.

Outline

01 Diffusion Parameterizations

02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance
Classifier-Free Guidance

Outline

01 **Diffusion Parameterizations**

02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance
Classifier-Free Guidance

Reparametrization of DDPM

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{1}{2\tilde{\beta}_t}\left\|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right\|^2\right]$$

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\cdot\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{1-\bar{\alpha}_t}\cdot\mathbf{x}_0$$

Forward Noising

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\cdot\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\cdot\boldsymbol{\epsilon}$$

Recover Clean Data

$$\mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\cdot\boldsymbol{\epsilon}}{\sqrt{\bar{\alpha}_t}}$$

Why reparametrize? Predict the Gaussian noise we add ourselves during training; compute the reverse mean analytically from this prediction.

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \boldsymbol{\epsilon}) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\cdot\mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1-\alpha_t)}{1-\bar{\alpha}_t}\cdot\left(\frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\cdot\boldsymbol{\epsilon}}{\sqrt{\bar{\alpha}_t}}\right)$$

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \boldsymbol{\epsilon}) = \frac{1}{\sqrt{\alpha_t}}\cdot\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{\alpha_t(1-\bar{\alpha}_t)}}\cdot\boldsymbol{\epsilon}$$

Reparametrization of DDPM

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{1}{2\tilde{\beta}_t}\left\|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right\|^2\right]$$

Target Mean

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{\alpha_t(1-\bar{\alpha}_t)}} \cdot \boldsymbol{\epsilon}$$

Model Mean

$$\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{\alpha_t(1-\bar{\alpha}_t)}} \cdot \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)$$

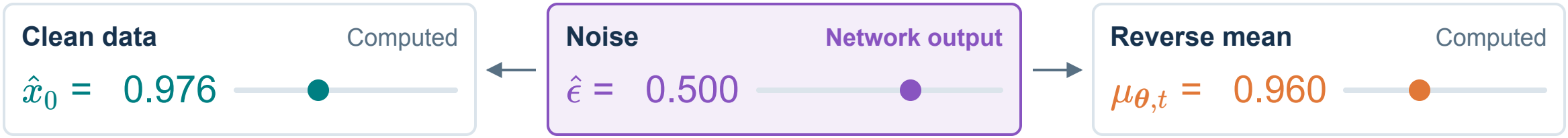
Noise-Prediction Loss

$$\begin{aligned}\mathcal{L}_t &= \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{(1-\alpha_t)^2}{2\tilde{\beta}_t\alpha_t(1-\bar{\alpha}_t)}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\right\|^2\right] \\ &= \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(0,\mathbf{I})}\left[\frac{(1-\alpha_t)^2}{2\tilde{\beta}_t\alpha_t(1-\bar{\alpha}_t)}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}\left(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}\right)\right\|^2\right]\end{aligned}$$

> Train the network to predict the Gaussian noise $\boldsymbol{\epsilon}$ used to construct $\mathbf{x}_t$.

Prediction Targets and Loss Functions

Linked predictions, one coordinate: $x_t = 1, \quad \alpha_t = 0.6, \quad \bar{\alpha}_t = 0.39$



Network output at $\mathbf{x}_t$		Training target	Squared error
Clean data	$\hat{\mathbf{x}}_{\boldsymbol{\theta},t}$	$\mathbf{x}_0$	$\ell_{x_0} = \lVert\hat{\mathbf{x}}_{\boldsymbol{\theta},t} - \mathbf{x}_0\rVert^2$
Noise	$\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}$	$\boldsymbol{\epsilon}$	$\ell_\epsilon = \lVert\boldsymbol{\epsilon}_{\boldsymbol{\theta},t} - \boldsymbol{\epsilon}\rVert^2$
Reverse mean	$\boldsymbol{\mu}_{\boldsymbol{\theta},t}$	$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0)$	$\ell_\mu = \lVert\boldsymbol{\mu}_{\boldsymbol{\theta},t} - \tilde{\boldsymbol{\mu}}_t\rVert^2$

For linked predictions, the same noise MSE, with $q = q(\mathbf{x}_t|\mathbf{x}_0)$:

$$\mathcal{L}_t^{(\epsilon)} = \mathbb{E}_q[\ell_\epsilon] = \mathbb{E}_q\left[\frac{\bar{\alpha}_t}{1-\bar{\alpha}_t}\ell_{x_0}\right] = \mathbb{E}_q\left[\frac{\alpha_t(1-\bar{\alpha}_t)}{(1-\alpha_t)^2}\ell_\mu\right]$$

Equivalent with these weights. Plain MSE in each parameterization weights timesteps differently.

Ho J. et al. Denoising Diffusion Probabilistic Models, 2020
Salimans T., Ho J. Progressive Distillation for Fast Sampling of Diffusion Models, 2022

Reparametrization of DDPM

$$\mathcal{L}_{\phi,\theta}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) - D_{\mathrm{KL}}\big(q(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T)\big) - \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} D_{\mathrm{KL}}\big(q(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)\|p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)\big)}_{\mathcal{L}_t}$$

$$\mathcal{L}_t = \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(0,\mathbf{I})}\left[\frac{(1-\alpha_t)^2}{2\tilde{\beta}_t\alpha_t(1-\bar{\alpha}_t)}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}\big(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}\big)\right\|^2\right]$$

- Dropping the time-dependent coefficient **changes the relative weights of noise levels**.
- The reconstruction term ($t = 1$) also uses noise prediction.

Simplified Objective

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t\sim\text{Uniform}\{1,\ldots,T\}}\mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(0,\mathbf{I})}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}\big(\sqrt{\bar{\alpha}_t}\cdot\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\cdot\boldsymbol{\epsilon}\big)\right\|^2$$

Outline

01 Diffusion Parameterizations

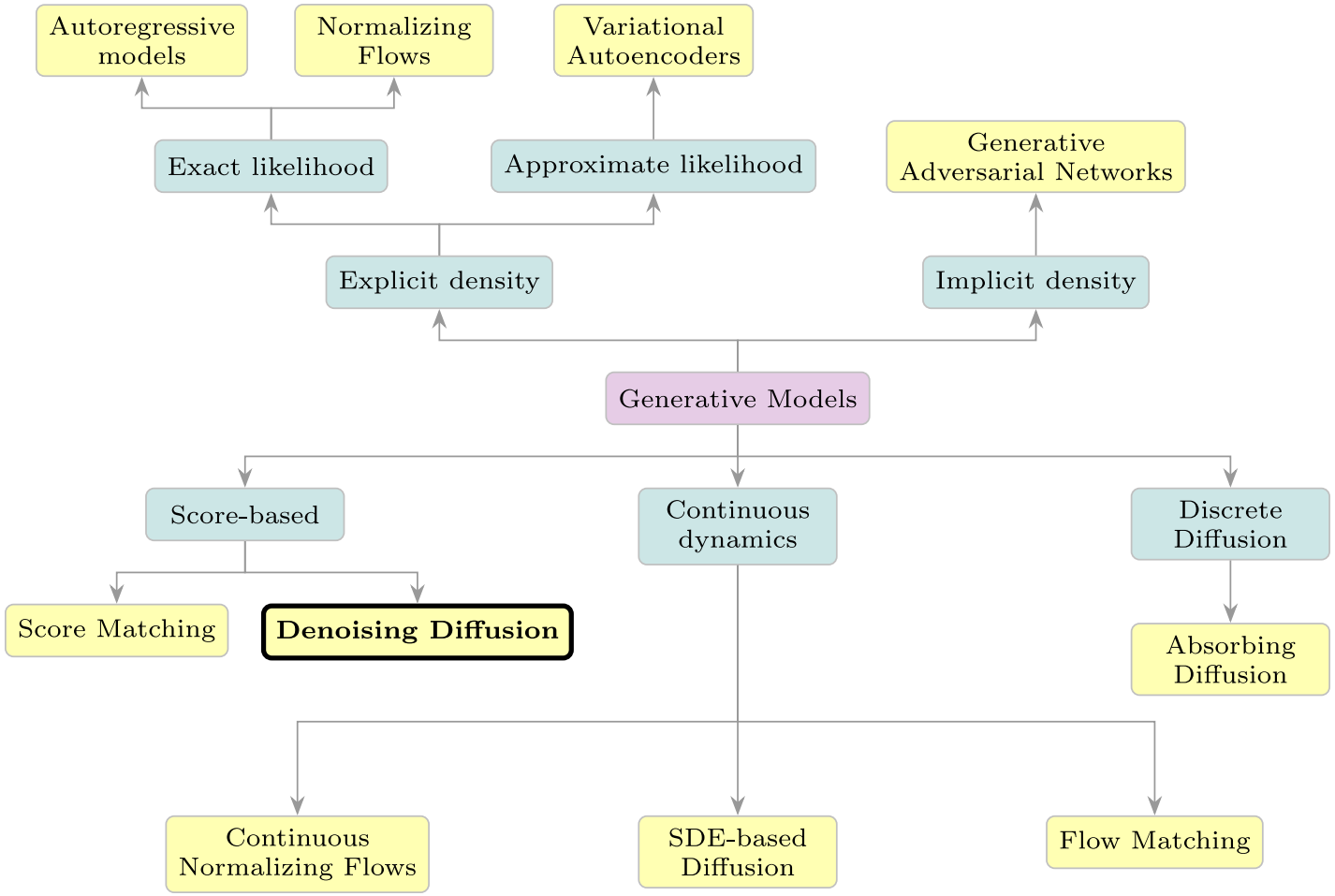
02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance
Classifier-Free Guidance

Generative Models Taxonomy



Denoising Diffusion Probabilistic Model (DDPM)

DDPM as a Hierarchical VAE

- **Encoder:** a fixed Gaussian Markov chain $q(\mathbf{x}_1, \ldots, \mathbf{x}_T | \mathbf{x}_0)$.
- **Latents:** $\mathbf{z} = (\mathbf{x}_1, \ldots, \mathbf{x}_T)$; each $\mathbf{x}_t$ has the same dimension as $\mathbf{x}_0$.
- **Decoder:** one Gaussian step $p_{\boldsymbol{\theta}}(\mathbf{x}_0 | \mathbf{x}_1)$.
- **Prior:** a learned Gaussian reverse chain from $p(\mathbf{x}_T) = \mathcal{N}(0, \mathbf{I})$ down to $\mathbf{x}_1$.

Fixed

Sample any $\mathbf{x}_t$ directly from $\mathbf{x}_0$:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\, \boldsymbol{\epsilon}.$$

Learned

Predict noise to obtain the reverse mean:

$$\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t) \quad \longrightarrow \quad \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t).$$

Training: one randomly chosen noise level per example.
Sampling: sequential reverse steps from noise to data.

Denoising Diffusion Probabilistic Model (DDPM)

Training

1. Sample $\mathbf{x}_0 \sim p_{\text{data}}(\mathbf{x})$, $t \sim \text{Uniform}\{1, \dots, T\}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$.
2. Compute noisy image $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\epsilon}$.
3. Minimize $\mathcal{L}_{\text{simple}} = \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\|^2$ by SGD.

Sampling (Ancestral)

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
2. For $t = T, \dots, 2$, compute the mean and sample the reverse step:

$$\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \frac{\mathbf{x}_t}{\sqrt{\alpha_t}} - \frac{1 - \alpha_t}{\sqrt{\alpha_t(1 - \bar{\alpha}_t)}} \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t),$$

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sqrt{\tilde{\beta}_t} \boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{I}).$$

3. Return the decoder mean $\hat{\mathbf{x}}_0 = \boldsymbol{\mu}_{\boldsymbol{\theta},1}(\mathbf{x}_1)$.

Denoising Diffusion as a Score-Based Generative Model

DDPM Objective

$$\mathcal{L}_t = \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(0,\mathbf{I})}\left[C_{1,t}\cdot\left\|\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)-\boldsymbol{\epsilon}\right\|_2^2\right]$$

$$= \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(0,\mathbf{I})}\left[C_{2,t}\cdot\left\|\frac{\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)}{\sqrt{1-\bar{\alpha}_t}}-\frac{\boldsymbol{\epsilon}}{\sqrt{1-\bar{\alpha}_t}}\right\|_2^2\right]$$

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\cdot\mathbf{x}_0, (1-\bar{\alpha}_t)\cdot\mathbf{I})$$

$$\nabla_{\mathbf{x}_t}\log q(\mathbf{x}_t|\mathbf{x}_0) = -\frac{\mathbf{x}_t-\sqrt{\bar{\alpha}_t}\cdot\mathbf{x}_0}{1-\bar{\alpha}_t} = -\frac{\boldsymbol{\epsilon}}{\sqrt{1-\bar{\alpha}_t}}.$$

Denoising Diffusion as a Score-Based Generative Model

We can reparameterize the model as:

$$\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = -\frac{\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)}{\sqrt{1-\bar{\alpha}_t}} = \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t).$$

This is the noise-to-score identity from **Tweedie's formula (Lecture 6)** at the MSE optimum.

$$\mathcal{L}_t = \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[C_{2,t} \cdot \left\| \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0) \right\|_2^2\right]$$

DDPM vs NCSN: Objectives

Write the conditional score error as

$$\ell_{\boldsymbol{\theta}}(t) = \mathbb{E}_{p_{\text{data}}(\mathbf{x}_0)} \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left\| \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0) \right\|^2 .$$

Same regression form, different weights

$$\mathcal{L} = \mathbb{E}_{t\sim U\{1,T\}} \left[w_t \, \ell_{\boldsymbol{\theta}}(t) \right] .$$

Objective	Gaussian perturbation	Score-loss weight w_t
DDPM simple loss	$\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$	$1 - \bar{\alpha}_t$
NCSN	$\mathbf{x}_0 + \sigma_t \boldsymbol{\epsilon}$	σ_t^2

The DDPM ELBO uses different weights. Replacing them changes the objective.

How can we choose training times without changing these weights?

Ho J., et al. Denoising Diffusion Probabilistic Models, 2020
Song Y., Ermon S. Generative Modeling by Estimating Gradients of the Data Distribution, 2019

Loss Weighting and Time Sampling

Let $\ell_{\boldsymbol{\theta}}(t)$ be the expected loss at step t, and fix weights w_t.

$$\mathcal{L} = \frac{1}{T}\sum_{t=1}^{T} w_t \ell_{\boldsymbol{\theta}}(t).$$

Sample training times with probability $r_t > 0$

$$\mathcal{L} = \mathbb{E}_{t\sim r}\left[\frac{w_t}{T r_t}\ell_{\boldsymbol{\theta}}(t)\right].$$

With the correction, the objective stays the same; gradient variance can change.

For two levels with $w_1 = w_2 = 1$ and $(r_1, r_2) = (0.9, 0.1)$:

$$0.9\frac{\ell_1}{2\cdot 0.9} + 0.1\frac{\ell_2}{2\cdot 0.1} = \frac{\ell_1 + \ell_2}{2}.$$

Without correction, $0.9\ell_1 + 0.1\ell_2$ is a **different objective**.

These probabilities select training levels, not generation steps.

DDPM and NCSN: Connecting Noise Coordinates

DDPM scales the clean signal as it adds noise:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

Divide by the signal coefficient

$$\underbrace{\frac{\mathbf{x}_t}{\sqrt{\bar{\alpha}_t}}}_{\tilde{\mathbf{x}}_t} = \mathbf{x}_0 + \underbrace{\sqrt{\frac{1-\bar{\alpha}_t}{\bar{\alpha}_t}}}_{\sigma_t}\boldsymbol{\epsilon}.$$

Here σ_t measures additive noise in the rescaled state.

Recover the NCSN noise form

$$\tilde{\mathbf{x}}_t = \mathbf{x}_0 + \sigma_t\boldsymbol{\epsilon}, \qquad q(\tilde{\mathbf{x}}_t \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_0, \sigma_t^2\mathbf{I}).$$

As $\bar{\alpha}_t$ decreases, the noise level σ_t increases.

The Gaussian perturbations are connected by rescaling. Next, compare their sampling rules: ancestral updates and Langevin dynamics.

Song J., Meng C., Ermon S. Denoising Diffusion Implicit Models, 2021
Song Y. et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020

Outline

01 Diffusion Parameterizations

02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance
Classifier-Free Guidance

DDPM vs NCSN: Sampling

Sampling (Ancestral)

$$\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$$

$$\begin{aligned}\mathbf{x}_{t-1} &= \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sigma_t \cdot \boldsymbol{\epsilon} \\ &= \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{\alpha_t(1-\bar{\alpha}_t)}} \cdot \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sigma_t \cdot \boldsymbol{\epsilon} \\ &= \frac{1}{\sqrt{1-\beta_t}} \cdot \mathbf{x}_t + \frac{\beta_t}{\sqrt{1-\beta_t}} \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sigma_t \cdot \boldsymbol{\epsilon}\end{aligned}$$

DDPM vs NCSN: Sampling

Sampling (Annealed Langevin Dynamics)

1. Sample $\mathbf{x}_T^0 \sim \mathcal{N}(0, \sigma_T^2 \mathbf{I}) \approx q(\mathbf{x}_T)$.
2. Update $\mathbf{x}_t^l$ via L steps of Langevin dynamics at each noise level σ_t:

$$\mathbf{x}_t^l = \mathbf{x}_t^{l-1} + \frac{\eta_t}{2} \cdot \mathbf{s}_{\boldsymbol{\theta},\sigma_t}(\mathbf{x}_t^{l-1}) + \sqrt{\eta_t} \cdot \boldsymbol{\epsilon}_t^l.$$

3. Update $\mathbf{x}_{t-1}^0 := \mathbf{x}_t^L$ and proceed to the next noise level σ_{t-1}.

DDPM vs NCSN: Summary

Summary

- Gaussian perturbations:
  - DDPM: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\epsilon}$;
  - NCSN: $\mathbf{x}_t = \mathbf{x}_0 + \sigma_t \cdot \boldsymbol{\epsilon}$.
  - Rescaling the DDPM state by $1/\sqrt{\bar{\alpha}_t}$ gives the NCSN noise form.
- Both use denoising score matching, with objective-specific noise-level weights.
- The sampling procedures differ:
  - Ancestral sampling in DDPM;
  - Annealed Langevin dynamics for NCSN;
  - Hybrid approaches that combine both updates are possible.

Kingma D. et al. Variational Diffusion Models, 2021
Song Y. et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020

DDIM: From Noise to Clean Data

Can we change the sampler without retraining the DDPM network?

Keep fixed: $q(\mathbf{x}_t \mid \mathbf{x}_0)$ at every noise level — the same noise-MSE training.

Change: the joint $q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)$ and its transitions, using a non-Markovian forward process.

At each noise level, the forward process gives:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}.$$

Predict the noise with $\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)$ and solve for the clean data:

$$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)}{\sqrt{\bar{\alpha}_t}}.$$

These two estimates exactly reconstruct the current state:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\hat{\mathbf{x}}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t).$$

DDIM idea: keep both estimates fixed for one transition and change only the weights, from t to $s < t$.

DDIM: Same Marginals, Different Transitions

Let $X \sim \mathcal{N}(0, 4)$ and $Z \sim \mathcal{N}(0, 1)$ be independent. Target: $Y \sim \mathcal{N}(0, 1)$.

Stochastic transition

$$Y = \frac{X}{4} + \frac{\sqrt{3}}{2} Z.$$

Variance: $\frac{4}{16} + \frac{3}{4} = 1.$

Deterministic transition

$$Y = \frac{X}{2}.$$

Variance: $\frac{4}{4} = 1.$

Both give the same distribution of Y, but different joint distributions of (X, Y).

The deterministic transition gets all its randomness from X.

Simply deleting Z from the first rule gives $Y = X/4 \sim \mathcal{N}(0, 1/4)$: the wrong variance.

Illustrative Gaussian example.

DDIM: Deterministic Sampling

Sampling (DDIM)

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$; choose a grid $T = t_K > \cdots > t_0 = 0$.
2. For $k = K, \ldots, 1$, set $t = t_k$, $s = t_{k-1}$ and compute:

$$\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t) \qquad \text{predict noise,}$$

$$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}} \qquad \text{estimate clean data,}$$

$$\mathbf{x}_s = \sqrt{\bar{\alpha}_s}\,\hat{\mathbf{x}}_0 + \sqrt{1-\bar{\alpha}_s}\,\hat{\boldsymbol{\epsilon}} \qquad \text{update state.}$$

3. Return $\mathbf{x}_0$.

Deterministic limit: fresh transition noise is zero ($\sigma_{t\to s} = 0$); randomness comes from $\mathbf{x}_T$.

Original training schedule: $s < t \Rightarrow \bar{\alpha}_s > \bar{\alpha}_t$: more signal, less noise.

Fewer steps: choose $K \ll T$; K steps require K network evaluations and no retraining.

Outline

01 Diffusion Parameterizations

02 Denoising Diffusion Probabilistic Model (DDPM)

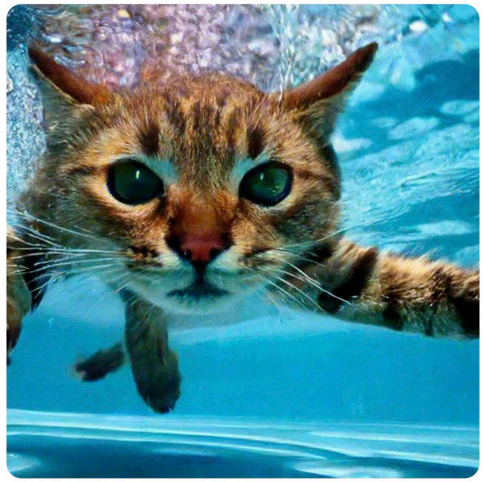
03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance
Classifier-Free Guidance

Guidance

- Up to now, we have focused on **unconditional** generative models $p_{\theta}(\mathbf{x})$.
- In practice, most generative models are **conditional** (in the diffusion era it is called guided): $p_{\theta}(\mathbf{x}|\mathbf{y})$.
- Here, $\mathbf{y}$ might denote a class label or **text** (as in text-to-image tasks).



Кот ныряет в бассейн, как ребенок на обложке альбома Nevermind, реалистично



рука человека с пятью пальцами, ни четыремя, ни шестью, а с 5 (пять) пальцами

Conditional Models

In practice, we're typically interested in learning conditional models (sampling from conditional distribution $p_{\text{data}}(\mathbf{x}|\mathbf{y})$).

- $\mathbf{y} = \emptyset$, $\mathbf{x}$ = image $\Rightarrow$ unconditional image model
- $\mathbf{y}$ = class label, $\mathbf{x}$ = image $\Rightarrow$ class-conditional image model
- $\mathbf{y}$ = text prompt, $\mathbf{x}$ = image $\Rightarrow$ text-to-image model
- $\mathbf{y}$ = image, $\mathbf{x}$ = image $\Rightarrow$ image-to-image model
- $\mathbf{y}$ = image, $\mathbf{x}$ = text $\Rightarrow$ image-to-text (image captioning) model
- $\mathbf{y}$ = English text, $\mathbf{x}$ = Russian text $\Rightarrow$ sequence-to-sequence model (machine translation)
- $\mathbf{y}$ = sound, $\mathbf{x}$ = text $\Rightarrow$ speech-to-text (automatic speech recognition) model
- $\mathbf{y}$ = text, $\mathbf{x}$ = sound $\Rightarrow$ text-to-speech model

Label Guidance

Label: Ostrich (10th ImageNet class)



VQ-VAE (Proposed) BigGAN deep

Razavi A., Oord A., et al. Generating Diverse High-Fidelity Images with VQ-VAE-2, 2019

Text Guidance

Prompt: Anna
SDXL fine-tuned on Disney princesses. Same initial noise; only γ changes.

$\gamma = 3.5$ $\gamma = 4.5$ $\gamma = 5.5$ $\gamma = 6.5$



Kasymov A., et al. AutoLoRA: AutoGuidance Meets Low-Rank Adaptation for Diffusion Models, 2024. Fig. 3, top row (selected columns). CC BY 4.0.

Guidance in Generative Models

How to make a guided model?

Instead of sampling from $p_{\boldsymbol{\theta}}(\mathbf{x})$, we sample from $p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{y})$.

Given **supervised** data $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$, we can treat $\mathbf{y}$ as an additional model input:

- $p_{\boldsymbol{\theta}}(x_j|\mathbf{x}_{1:j-1}, \mathbf{y})$ for AR models;
- Encoder $q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}, \mathbf{y})$ and decoder $p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z}, \mathbf{y})$ for VAEs;
- $G_{\boldsymbol{\theta}}(\mathbf{z}, \mathbf{y})$ for NFs and GANs;
- $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y})$ for DDPMs.

Challenge

- Empirically, images sampled with this procedure do not fit well enough to the desired label $\mathbf{y}$.
- Being able to control the strength of guidance is especially valuable.

Outline

01 Diffusion Parameterizations

02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance

Classifier-Free Guidance

Classifier Guidance

DDPM Sampling

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
2. Denoise the image (unconditional generation):

$$\begin{aligned}\mathbf{x}_{t-1} &= \frac{1}{\sqrt{1-\beta_t}} \cdot \mathbf{x}_t + \frac{\beta_t}{\sqrt{1-\beta_t}} \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sigma_t \cdot \boldsymbol{\epsilon} \\ &= \frac{1}{\sqrt{1-\beta_t}} \cdot \mathbf{x}_t + \frac{\beta_t}{\sqrt{1-\beta_t}} \cdot \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \sigma_t \cdot \boldsymbol{\epsilon}\end{aligned}$$

Guided Generation

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \cdot \mathbf{x}_t + \frac{\beta_t}{\sqrt{1-\beta_t}} \cdot \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) + \sigma_t \cdot \boldsymbol{\epsilon}$$

What is the link between $\nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t)$ and $\nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y})$?

Classifier Guidance: Guided Score Function

Guided Generation

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \cdot \mathbf{x}_t + \frac{\beta_t}{\sqrt{1-\beta_t}} \cdot \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) + \sigma_t \cdot \boldsymbol{\epsilon}$$

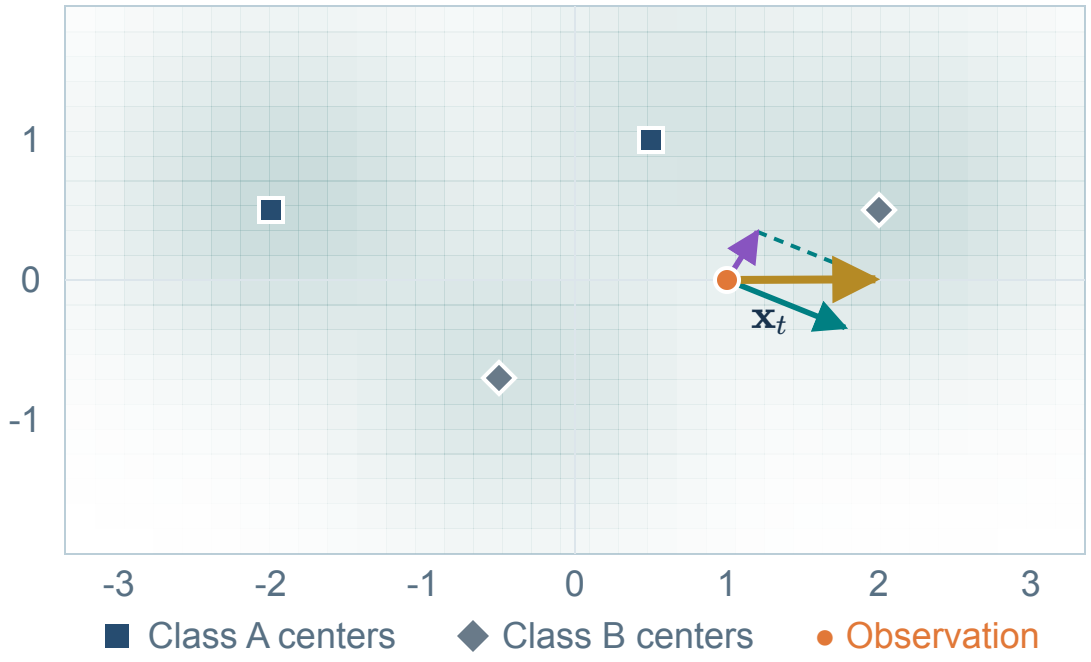
Guided Generation

$$\begin{aligned}\nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) &= \nabla_{\mathbf{x}_t} \log \left(\frac{p_{\boldsymbol{\theta}}(\mathbf{x}_t) p(\mathbf{y}|\mathbf{x}_t)}{p(\mathbf{y})} \right) \\ &= \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) \\ &= \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) = \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})\end{aligned}$$

Classifier Guidance: Which Way Does It Push?

Four Gaussian components, two classes, one fixed noise level.

Fixed observation · target class B · γ = 3.



Gaussian component centers and the marginal density.

Unconditional score

$\mathbf{s}_t(\mathbf{x}_t)$ (1.08, 1.85)

Classifier contribution

$\gamma \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)$ (4.18, -1.83)

Guided score

$\mathbf{s}_t^{\gamma}(\mathbf{x}_t, \mathbf{y})$ (5.25, 0.03)

$p(\mathbf{y}|\mathbf{x}_t)$ = 0.648

Exact toy scores at a fixed noise level. At this point, the common scale is fixed across γ and classes.

Guidance adds a class-directed correction to the unconditional score.

Classifier Guidance: Guidance Scale

Guided Score Function

$$\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) = \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)$$

- Let us assume $\mathbf{y}$ is a class label.
- $p(\mathbf{y}|\mathbf{x}_t)$ is a classifier for noisy inputs.
- $p(\mathbf{y}|\mathbf{x}_t)$ is responsible for model guidance.

Guidance Scale

It is a natural idea to scale up the contribution of the guidance

$$\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) = \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)$$

- The guidance scale γ adjusts the strength of classifier guidance.
- $\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$ is not the true guided score function $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$.

Classifier Guidance: Distribution Sharpening

Scaled Guided Score Function

$$\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) = \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)$$

Scaled Conditional Distribution at a Fixed Noise Level

$$\begin{aligned}
\nabla^{\gamma}_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) &= \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) \\
&= \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)^{\gamma} \\
&= \nabla_{\mathbf{x}_t} \log \left(\frac{p_{\boldsymbol{\theta}}(\mathbf{x}_t) p(\mathbf{y}|\mathbf{x}_t)^{\gamma}}{Z} \right)
\end{aligned}$$

Note: Increasing γ sharpens $p(\mathbf{y}|\mathbf{x}_t)$, increasing the contrast

$$\hat{p}(\mathbf{y}|\mathbf{x}_t) \propto p(\mathbf{y}|\mathbf{x}_t)^{\gamma}.$$

Very large γ can reduce diversity and introduce artifacts.

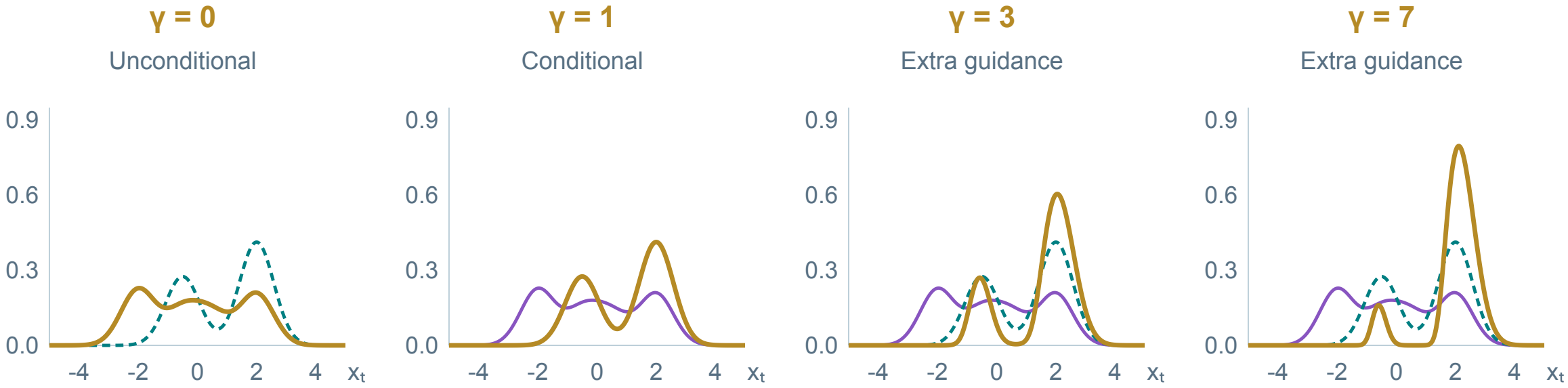
Dhariwal P., Nichol A. Diffusion Models Beat GANs on Image Synthesis, 2021
Sadat S. et al. Eliminating Oversaturation and Artifacts of High Guidance Scales in Diffusion Models, 2025

Guidance Scale Changes the Density

$$q_\gamma(x_t|y) \propto q(x_t)\, p(y|x_t)^\gamma \qquad \text{at a fixed noise level.}$$

Same one-dimensional toy and target class B; only γ changes.

$q(x_t)$ $q(x_t|y)$ · dashed $q_\gamma(x_t|y)$



Increasing γ shifts mass toward regions with higher classifier confidence.

A 1D version of the Gaussian-mixture toy, at a fixed noise level. This plot describes a density at that level.

Classifier Guidance: Overview

Training

1. Train the DDPM as before.
2. Train an additional classifier $p(\mathbf{y}|\mathbf{x}_t)$ on noisy data (time-dependent).

Sampling (Guided)

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
2. Denoise with the scaled guided score function:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \cdot \mathbf{x}_t + \frac{\beta_t}{\sqrt{1-\beta_t}} \cdot \mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) + \sigma_t \cdot \boldsymbol{\epsilon}.$$

Outline

01 Diffusion Parameterizations

02 Denoising Diffusion Probabilistic Model (DDPM)

03 From DDPM to DDIM

04 Model Guidance

Classifier Guidance

Classifier-Free Guidance

Classifier-Free Guidance

- The previous approach relies on training an additional classifier $p(\mathbf{y}|\mathbf{x}_t)$ for noisy images.
- We now introduce a method to sidestep this requirement.

$$\nabla^{\gamma}_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) = \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t)$$

Bayes theorem

$$\begin{aligned}\nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) &= \nabla_{\mathbf{x}_t} \log \left(\frac{p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) p(\mathbf{y})}{p_{\boldsymbol{\theta}}(\mathbf{x}_t)} \right) \\ &= \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) - \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t)\end{aligned}$$

Classifier-Free Guidance

Scaled Guided Score Function

$$
\begin{aligned}
\nabla^{\gamma}_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) &= \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) \\
&= \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \gamma \cdot \big(\nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) - \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t)\big) \\
&= (1-\gamma) \cdot \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y})
\end{aligned}
$$

Classifier-Free Guidance: Formulation

Scaled Guided Score Function

$$\nabla^{\gamma}_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y}) = (1-\gamma) \cdot \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{y})$$

$$\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) = (1-\gamma) \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$$

Naive training approach

- Train an unguided score function model $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t)$.
- Train a guided score function model $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$.
- Use their affine combination at inference.

Sampling (Guided)

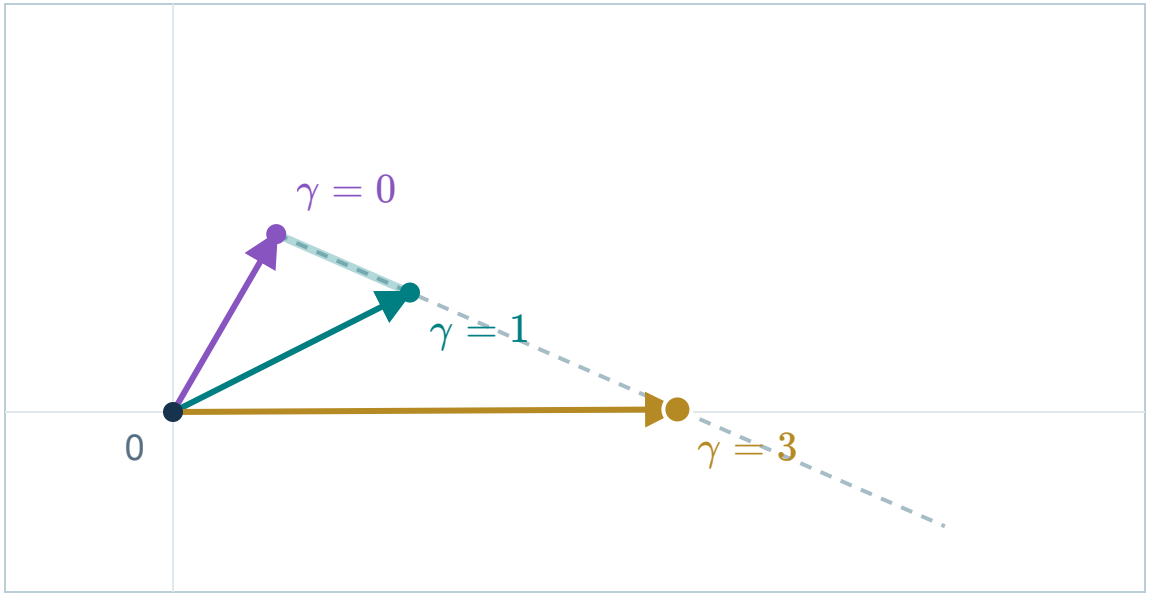
$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \cdot \mathbf{x}_t + \frac{\beta_t}{\sqrt{1-\beta_t}} \cdot \mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) + \sigma_t \cdot \boldsymbol{\epsilon}$$

How to avoid training two separate score function models?

CFG: Interpolation and Extrapolation

$$\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t} = \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \varnothing) + \gamma\big(\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) - \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \varnothing)\big)$$

Same two scores: $\gamma = 0$, $\gamma = 1$ and $\gamma = 3$.



Score space · endpoints of vectors

The two base scores are fixed; γ moves the guided endpoint along their line.

$\gamma = 3.0$ · Extrapolation

$\gamma = 0$

$\mathbf{s}^0_t = \mathbf{s}_t(\mathbf{x}_t, \varnothing)$

$\gamma = 1$

$\mathbf{s}^1_t = \mathbf{s}_t(\mathbf{x}_t, \mathbf{y})$

$\gamma > 1$

Continue beyond the conditional endpoint.

$\varnothing$ denotes no condition. Analytic toy scores at one fixed observation and noise level.

For $\gamma > 1$, CFG extrapolates beyond the conditional score.

Classifier-Free Guidance

$$\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) = (1-\gamma) \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$$

CFG

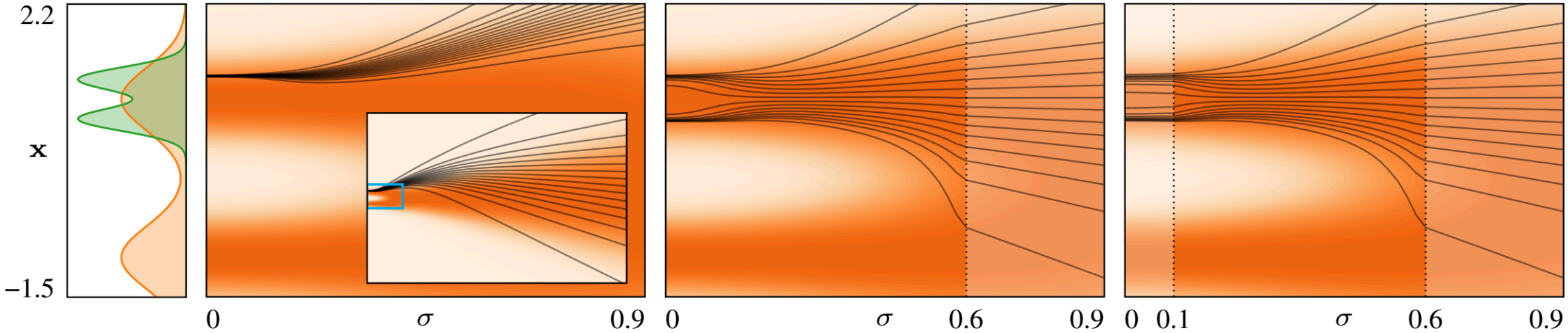
1. Introduce the "absence of conditioning" label $\mathbf{y} = \varnothing$.
2. Identify the unguided score function $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \varnothing)$.
3. Train a single model $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$ on **supervised** data, dropping the label $\mathbf{y}$ with some fixed probability (simulating $\mathbf{y} = \varnothing$).
4. At inference, evaluate the model twice to obtain $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \varnothing)$ and $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$.

Sampling (Guided)

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \cdot \mathbf{x}_t + \frac{\beta_t}{\sqrt{1-\beta_t}} \cdot \mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) + \sigma_t \cdot \boldsymbol{\epsilon}.$$

Classifier-Free Guidance: Guidance Interval

Apply extra guidance only at intermediate noise levels.



(a) PDFs (b) Guidance everywhere (c) Guidance at $\sigma \in (0, 0.6]$ (d) Guidance at $\sigma \in (0.1, 0.6]$

Toy example ($\gamma = 6$): guidance everywhere drops a mode; limiting the interval restores both.

Outside the interval, $\gamma_t = 1$: **conditioning remains**. Sampling runs **right to left** ($\sigma \downarrow$).

Kynkäänniemi T. et al. Applying Guidance in a Limited Interval Improves Sample and Distribution Quality in Diffusion Models, 2024. Fig. 2.

Classifier-Free Guidance: Guidance Distillation

Learn the guided prediction in a single model evaluation.

Teacher: two evaluations

$$\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \varnothing), \quad \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$$

Combine predictions using γ.

distill $\longrightarrow$

Student: one evaluation

$$\mathbf{s}_{\boldsymbol{\phi},t}(\mathbf{x}_t, \mathbf{y}, \gamma)$$

Pass γ as an extra input.

$$\mathbf{s}_{\boldsymbol{\phi},t}(\mathbf{x}_t, \mathbf{y}, \gamma) \approx (1-\gamma)\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \varnothing) + \gamma \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$$

Training

1. Sample $(\mathbf{x}_0, \mathbf{y}) \sim p_{\text{data}}$, $t \sim \text{Uniform}\{1, \dots, T\}$, $\gamma \sim \text{Uniform}[\gamma_{\min}, \gamma_{\max}]$.
2. Sample $\mathbf{x}_t \sim q(\mathbf{x}_t|\mathbf{x}_0)$ and compute the **fixed teacher target** $\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$.
3. Update only $\boldsymbol{\phi}$ to minimize $\mathcal{L} = \|\mathbf{s}_{\boldsymbol{\phi},t}(\mathbf{x}_t, \mathbf{y}, \gamma) - \mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})\|_2^2$.

Sampling

1. Evaluate the student once per step; keep the sampler and step count. Used in **FLUX.2 [dev]**.

Meng C. et al. On Distillation of Guided Diffusion Models, 2023.
Black Forest Labs. FLUX.2 [dev] model card: guidance distillation.

Summary

- Reverse-mean, clean-data and noise predictions are convertible; unweighted losses assign different weights to noise levels.
- DDPM replaces ELBO weights with a simple noise-prediction loss and generates through Gaussian reverse steps.
- DDPM and NCSN are weighted denoising score matching in rescaled noise coordinates; corrected time sampling preserves a chosen objective.
- DDPM and NCSN use different samplers; DDIM reuses the DDPM network on a shorter sampling grid.
- Guidance makes generation controllable through labels or text prompts.
- Classifier guidance uses an auxiliary noisy-data classifier; classifier-free guidance does not.
- Guidance intervals can improve the quality-diversity trade-off; guidance distillation reduces evaluations per step.